

BORDER OPTICS — POORI RESEARCH SAMJHO

**PDF 10 of 22 — Acquisition: Multi-Year, Pre-Treatment &
Triangulation**

7 Files, Ek Saath — Poori Line By Line

`extract_multiyear_satellite_data.py · extract_pretreatment_baseline.py ·
extract_dw_pretreatment_baseline.py · extract_sar_backscatter.py ·
extract_dynamicworld_built.py · extract_building_footprints.py ·
extract_buffer_sensitivity_data.py`

Sakshi ke liye, Sakshi ki apni research par. Ye is project ki acquisition-side ki aakhri, sabse badi PDF hai — 7 files, jo saari PDF 8/9 ki wahi neev, tested machinery (500m buffer, QA60 cloud-mask, before/after NDBI/lights) reuse karke, poori research ko naye tareekon se stress-test karti hain: kya trend sirf do points ka artifact hai (multi-year)? Kya treatment se pehle bhi koi farak tha (pre-treatment placebo, do baar — NDBI/lights ke liye aur Dynamic World ke liye)? Kya ek bilkul alag satellite (radar) bhi wahi kahani batata hai (SAR)? Kya ek alag algorithm bhi (Dynamic World)? Kya asli buildings bhi wahi kahani dikhati hain (building footprints)? Aur kya buffer ka size khud result badal deta hai (buffer sensitivity)?

Shuru Karte Hain — Ye Saati Files Milkar Kya Kar Rahi Hain

PDF 8 aur PDF 9 mein humne dekha ki is research ka core measurement kaise hota hai — 500-meter buffer, QA60 (ya SCL) se cloud hatana, NDBI aur night-lights nikaalna. Lekin ek acha researcher sirf ek measurement lekar khush nahi ho jaata — wo poochta hai: 'agar main isi cheez ko thoda alag tareeke se measure karoon, kya wahi jawaab aayega?' Yahi is PDF ka poora kaam hai. Saati files, alag-alag angle se, wahi ek core sawaal poochti hain — 'kya VVP-I ne satellite se dikhne wala built-up-area badla' — aur har baar, jawaab ko dobara test karti hain.

EXAMPLE — Ek Hi Mureed Ko Saat Alag Doctors Se Check Karwana

PDF 9 mein humne 'doosri opinion' ka concept dekha tha — ek doosra doctor, alag measurement-technique se. Ye PDF us idea ko aur aage le jaata hai: ab sirf ek doosra doctor nahi, poore saat alag specialists hain, har ek apne apne angle se check kar raha hai. Ek time-ka-doctor (multi-year) poochta hai 'kya ye trend hai ya sirf do din ka farak?' Do 'pehle se hi healthy the' doctors (dono pre-treatment placebo tests) poochte hain 'kya patient pehle se hi is direction mein badal raha tha, treatment se pehle hi?' Ek alag-machine-wala doctor (SAR, radar) poochta hai 'agar main bilkul alag equipment use karoon, kya wahi result aayega?' Ek alag-training-wala doctor (Dynamic World) poochta hai 'agar main isi X-ray ko alag tarike se padhoon (alag algorithm), kya wahi baat nikalti hai?' Ek 'zameen par jaake dekho' doctor (building footprints) poochta hai 'chalo, asli buildings count karte hain, satellite-proxy se nahi.' Aur ek 'zoom in/out karke dekho' doctor (buffer sensitivity) poochta hai 'agar main apna measurement-area thoda bada ya chhota karoon, kya wahi baat dikhti hai?'

In saati files ka ek zaroori, practical common-point hai: ye sab, kisi na kisi hadd tak, PDF 8 ki `extract_satellite_data.py` aur `extract_control_satellite_data.py` ka hi code reuse karti hain — `init_ee()`, `mask_s2_clouds()`, `ndbi_for_period()`, `lights_for_period()`, aur ek 'coordinate se resume karo, master-list se mismatch ho toh fresh shuru karo' pattern jo control-group ki naturally-badalti-rehne-wali list ke liye zaroori hai. Isliye is PDF mein hum pehle in shared pieces ko ek jagah, poori detail mein samjhenge — aur uske baad har file mein sirf wahi cheez dekhenge jo *genuinely naya* hai us file mein, purani cheez ko dobara line-by-line dohraye bina.

History Ek Nazar Mein — Ek Bug Jo Teen Jagah Pakda Gaya

Is PDF ki sabse important connecting-kahani Development Log Entry 37 mein hai. Jab control-village list ek district-verification fix se 732 se 721 ho gayi (Sikmim/North district ka permanent gap alag rakh ke), teen extraction scripts — `extract_dynamicworld_built.py`, `extract_sar_backscatter.py`, aur

`extract_building_footprints.py` — mein ek hi tarah ka resume-logic bug mila: ye scripts, control-group ke liye, purani checkpoint file ko bina kisi mismatch-check ke seedha load kar lete the — matlab, agar village-list badal bhi jaaye, ye scripts chup-chaap purani, stale 732-village checkpoint hi use karte reh jaate. Ye poori PDF isi 'ek hi bug, teen jagah' kahani ko, code ke exact un hisso ke saath dekhegi jahan ye tha.

Shared Skeleton — Wo Cheezein Jo Har File Mein Reuse Hoti Hain

Neeche diye gaye char pieces — `init_ee()`, `mask_s2_clouds()`, `ndbi_for_period()`, `lights_for_period()` — PDF 8 mein already poori tarah, line-by-line, purpose+syntax ke saath explain ho chuke hain. Ye char functions, is PDF ki saati files mein se panch mein (multi-year, pre-treatment, SAR ke alawa — SAR aur Dynamic World apne khud ke alag functions rakhte hain jo aage dekhenge), *character-by-character* wahi hain jo PDF 8 mein the — ek line bhi alag nahi. Isliye unhe yahan dobara poori detail mein nahi doharayenge — sirf ek chhota reminder, taaki tu confirm kar sake ki ye wahi hi hain.

```
def init_ee():
    project = os.environ.get("EE_PROJECT")
    try:
        ee.Initialize(project=project)
    except Exception:
        ee.Authenticate()
        ee.Initialize(project=project)

def mask_s2_clouds(image):
    qa = image.select("QA60")
    cloud_bit_mask = 1 << 10
    cirrus_bit_mask = 1 << 11
    mask = qa.bitwiseAnd(cloud_bit_mask).eq(0).And(qa.bitwiseAnd(cirrus_bit_mask).eq(0))
    return image.updateMask(mask).divide(10000)
```

Purpose: `init_ee()` Earth Engine se connect karta hai (PDF 8 mein poori detail mein dekha — `EE_PROJECT` environment-variable, try/except-authenticate pattern). `mask_s2_clouds()` QA60 bitmask se cloudy pixels hataata hai (bits 10/11, cloud aur cirrus). Ye is project ki sabse zyada reuse hone wali do functions hain — is PDF ki 7 files mein se 5 mein bilkul yehi code hai. Syntax: Dono function-definitions, PDF 8 se word-for-word same. Agar tu unki syntax bhool gayi hai, PDF 8 ke `init_ee()`/`mask_s2_clouds()` sections dobara dekh sakti hai.

```
def ndbi_for_period(buffered_geom, start, end):
    collection = (
        ee.ImageCollection(S2_COLLECTION)
        .filterBounds(buffered_geom)
        .filterDate(start, end)
        .map(mask_s2_clouds)
    )
    count = collection.size().getInfo()
    if count == 0:
        return None, 0
    composite = collection.median()
```

```

ndbi_image = composite.normalizedDifference(["B11", "B8"]).rename("NDBI")
stats = ndbi_image.reduceRegion(
    reducer=ee.Reducer.mean(), geometry=buffered_geom, scale=10, maxPixels=1e9,
).getInfo()
return stats.get("NDBI"), count

def lights_for_period(buffered_geom, start, end):
    collection =
ee.ImageCollection(VIIRS_COLLECTION).filterBounds(buffered_geom).filterDate(start,
end)
    count = collection.size().getInfo()
    if count == 0:
        return None, 0
    composite = collection.select("avg_rad").mean()
    stats = composite.reduceRegion(
        reducer=ee.Reducer.mean(), geometry=buffered_geom, scale=500, maxPixels=1e9,
    ).getInfo()
    return stats.get("avg_rad"), count

```

Purpose: `ndbi_for_period()` aur `lights_for_period()` — dono, PDF 8 mein poori detail mein dekhe gaye. NDBI: median composite, $(B11-B8)/(B11+B8)$ formula, 10m scale. Lights: mean composite, VIIRS avg_rad band, 500m scale (VIIRS ka apna coarser native resolution). Dono zero-image null-safe return karte hain. Syntax: Method-chaining, `.map()` se function-per-image apply karna, `reduceRegion(reducer=..., geometry=..., scale=..., maxPixels=...)` — sab PDF 8 mein already dekha.

Coordinate-Based Resume, Mismatch-Check Ke Saath — Doosra Shared Piece

PDF 8 ki `extract_control_satellite_data.py` mein humne ek pattern dekha tha: control-village list, jab bhi `select_control_villages.py` dobara chalti hai, poori tarah regenerate ho sakti hai — isliye purani checkpoint file ko bina check kiye trust nahi kiya ja sakta. Fix: current village-list ke coordinates aur checkpoint file ke coordinates ka set compare karo; agar match ho, resume karo; agar na ho, checkpoint ko ignore karke fresh shuru karo. Ye pattern is PDF ki teen files (SAR, Dynamic World, building-footprints) mein bhi hai — aur yahi wo jagah hai jahan Entry 37 ka bug tha.

```

villages = pd.read_csv(VILLAGE_PATHS[group])
for col in outcome_cols:
    if col not in villages.columns:
        villages[col] = None

if os.path.exists(out_path):
    existing = pd.read_csv(out_path)
    current_coords = set(zip(villages["latitude"].round(6), villages["longitude"].round(6)))
    existing_coords = (
        set(zip(existing["latitude"].round(6), existing["longitude"].round(6)))

```

```

        if {"latitude", "longitude"}.issubset(existing.columns) else set()
    )
    if existing_coords and existing_coords == current_coords:
        lookup = existing.copy()
        lookup["_coord_key"] = list(zip(lookup["latitude"].round(6),
lookup["longitude"].round(6)))
        lookup_dict = lookup.set_index("_coord_key")[
            [c for c in outcome_cols if c in lookup.columns]
        ].to_dict("index")
        villages_keys = list(zip(villages["latitude"].round(6), villages["longitude"].round(6)))
        for i, key in zip(villages.index, villages_keys):
            values = lookup_dict.get(key)
            if values:
                for col in outcome_cols:
                    if col in values:
                        villages.at[i, col] = values[col]
        print(f"Resuming from existing checkpoint: {out_path} (matched by coordinate,
{len(existing)} rows)")
    else:
        print(f"Checkpoint at {out_path} doesn't match current village list -- starting
fresh.")

```

Purpose: Ye poora block, jo is PDF ki teen files mein appear hoga, sabse pehle village-list *current* file se padhta hai (checkpoint se nahi) — matlab, master-list hi source-of-truth hai. Missing outcome-columns khaali bhar diye jaate hain. Fir, agar checkpoint file exist karti hai, uske coordinates aur current list ke coordinates, dono ko rounded (6 decimal) latitude/longitude pairs ke sets mein badla jaata hai. Agar dono sets *exactly* match karte hain, tabhi resume hota hai — coordinate-key se ek lookup-dictionary banakar, purani values wapas current dataframe mein daali jaati hain. Agar match nahi karte (village-list badal chuki hai), ek clear message print hota hai aur poora extraction fresh shuru hota hai — koi bhi stale data chup-chaap reuse nahi hoti. Syntax: `set(zip(col1.round(6), col2.round(6)))` — do columns se coordinate-pairs ka ek Python set banana, taaki fast comparison ho sake (`==` se do sets compare karna, order-independent). `{"latitude", "longitude"}.issubset(existing.columns)` — check karna ki dono column-naam existing dataframe mein hain ya nahi, agar nahi toh empty set (safe default). `lookup.set_index("_coord_key")[cols].to_dict("index")` — dataframe ko ek dictionary mein badalna jiski key coordinate-pair hai aur value baaki columns ka dictionary. `lookup_dict.get(key)` — safe dictionary lookup, na milne par None.

EXAMPLE — Entry 37 — Wahi Fix, Lekin Do Jagah Missing Tha

Ye exact resume-pattern PDF 8 ki `extract_control_satellite_data.py` mein sabse pehle likha gaya. Jab control-list 732 se 721 ho gayi (district-verification fix, Sikkim/North ka permanent gap), Entry 37 mein check kiya gaya ki har control-dependent script sahi tarike se react karta hai ya nahi. `extract_control_satellite_data.py` ne sahi kaam

kiya — 'Checkpoint doesn't match... starting fresh' print karke, saari 721 villages fresh se process ki. Lekin `extract_dynamicworld_built.py` aur `extract_sar_backscatter.py` — dono ne '732 villages to process' print kiya, matlab dono chup-chaap purani, 732-village checkpoint file hi load kar rahe the, koi mismatch-check tha hi nahi unme. Root cause: ye dono scripts, is upar diye gaye poore coordinate-mismatch-check block ke bina likhi gayi thi — bas seedha `if os.path.exists(out_path): existing = pd.read_csv(out_path)` karke use kar leti thi, bina check kiye ki wo current list se match karta hai ya nahi. Ye is PDF mein aage dono files (SAR aur Dynamic World sections) mein dobara aayega, jahan hum exact fix dekhenge.

File 1 — extract_multiyear_satellite_data.py: Do Points Nahi, Teen

Docstring, Imports, Aur Constants

```
"""Adds a 2023 time point between the 2021/2025 pair so a three-point trend
can be fit instead of a single before-after difference.
"""
```

```
import argparse
import os
import time

import ee
import pandas as pd
from dotenv import load_dotenv

load_dotenv()
```

Purpose: Docstring seedha, ek line mein poora point bata deta hai: 2021-vs-2025 ka do-point difference, ek anomalous saal se hi driven ho sakta hai — agar beech mein ek teesra point (2023) mil jaaye, tab ek genuine trend-line fit ki ja sakti hai, sirf ek difference nahi. Imports bilkul wahi jo poore acquisition-module mein consistent hain. **Syntax:** Standard imports, jaisa poore project mein dekha.

```
VILLAGES_PATH = "data/processed/border_optics_master_villages.csv"
BUFFER_RADIUS_M = 500

S2_COLLECTION = "COPERNICUS/S2_SR_HARMONIZED"
VIIRS_COLLECTION = "NOAA/VIIRS/DNB/MONTHLY_V1/VCMSLCFG"

YEARS = [2021, 2023, 2025]

WINDOW_MONTHS = {
    "full_year": {"out_path": "data/processed/border_optics_multiyear_fullyear.csv"},
    "summer": {"out_path": "data/processed/border_optics_multiyear_summer.csv"},
}
```

Purpose: Wahi master village-list, wahi 500m buffer, wahi do collections — sab PDF 8 se parichit. Naya yahan YEARS = [2021, 2023, 2025] hai — teen saal ki ek list, jo aage loop mein iterate hogi. WINDOW_MONTHS, PDF 8 ki WINDOWS dictionary jaisi hi hai, lekin simpler — is baar sirf out_path rakhta hai, kyunki before/after date-tuples yahan hardcoded nahi, balki neeche ek function se dynamically banenge (kyunki teen saal hain, do nahi). **Syntax:** List literal, teen integers ka. Nested dictionary, do window-options ke liye, har ek sirf ek key ("out_path") rakhta hai.

year_range() — Ek Saal Ko Date-Range Mein Badalna

```
def year_range(window_key, year):
    if window_key == "full_year":
        return f"{year}-01-01", f"{year + 1}-01-01"
    else: # summer: June 1 - Sep 30, season-matched
        return f"{year}-06-01", f"{year}-10-01"
```

Purpose: Ye is file ka pehla genuinely naya function hai. PDF 8 mein before/after date-ranges directly hardcoded the (jaise ("2021-01-01", "2022-01-01")), kyunki sirf do fixed saal the. Yahan teen saal hain, aur teeno ke liye wahi pattern chahiye — isliye ek function likha gaya jo koi bhi saal (aur window-type) lekar, uska sahi date-range bana deta hai. "full_year" ke liye, wo saal ka January 1 se agle saal ke January 1 tak (matlab poora ek calendar year). "summer" ke liye, usi saal ka June 1 se October 1 tak. Syntax: F-strings ke andar arithmetic — `f"{year + 1}-01-01"` — variable year ko pehle 1 se add kiya jaata hai, fir uska result string mein embed hota hai. Function do values ek saath return karta hai (return a, b), jo ek tuple ban jaata hai, tuple-unpacking se aage receive hoga.

EXAMPLE — Ye Function PDF 8 Mein Kyun Nahi Tha

Socho agar tujhe teen saal ke liye date-ranges chahiye hon, bina is function ke — tujhe har jagah `if year == 2021: start = "2021-01-01"...` `elif year == 2023: ...` jaisa likhna padta, teen baar. Is function ne wahi repetitive logic ek jagah, ek formula mein samet diya — koi bhi saal do, sahi date-range wapas milegi, bina copy-paste ke. Ye function-writing ka poora fayda hai: jab koi pattern repeat ho raha ho, use ek function bana do.

extract_window() — Har Gaon Ke Liye Teen Saal Ka Data

```
def extract_window(window_key, checkpoint_every=10):
    out_path = WINDOW_MONTHS[window_key]["out_path"]
    villages = pd.read_csv(VILLAGES_PATH)

    for year in YEARS:
        for prefix in ["ndbi", "ndbi_n", "lights", "lights_n"]:
            col = f"{prefix}_{year}"
            if col not in villages.columns:
                villages[col] = None
```

Purpose: Ye function is file ka orchestrator hai. Path nikaalna, village-list padhna — seedha, PDF 8 jaisa. Naya hissa: ek *nested loop* jo har saal (2021, 2023, 2025) ke liye, char alag prefixes (ndbi, ndbi_n, lights, lights_n) se column-naam banata hai — jaise ndbi_2021, ndbi_n_2021, lights_2021, aur waise hi 2023 aur 2025 ke liye. Matlab, teen saal × char prefixes = 12 columns, sab dynamically naam banaakar. Har column, agar pehle se nahi hai, khaali (None) se bhar di jaati hai. **Syntax:** Do nested for loops — bahar wala YEARS par, andar wala ek fixed list of prefixes par. f"{prefix}_{year}" — f-string mein do variables ko jodkar ek naya column-naam banana, jaise "ndbi" + "_" + "2021" = "ndbi_2021".

```
for i, row in villages.iterrows():
    already_done = all(pd.notna(row.get(f"ndbi_{y}")) for y in YEARS) and \
        all(pd.notna(row.get(f"lights_{y}")) for y in YEARS)
    if already_done:
        continue

    point = ee.Geometry.Point([row["longitude"], row["latitude"]])
    buffered_geom = point.buffer(BUFFER_RADIUS_M)
```

Purpose: Skip-check yahan thoda alag hai PDF 8 se — kyunki ab char nahi, chhah outcome-columns (teen saal × NDBI, teen saal × lights, image-counts alag se) check karne hain, isliye ye do all(...) generator-expressions ko and se jodta hai: 'har saal ka NDBI bhara hai aur har saal ka lights bhara hai, tabhi ye gaon done hai.' Fir wahi point/buffer banana, jaisa har jagah. **Syntax:** Do all(pd.notna(...) for y in YEARS) generator-expressions, ek and se jode hue. Line ke aakhir mein backslash (\) — Python mein ek statement ko agli line par continue karne ka tareeka, jab wo brackets ke andar na ho (yahan and ke baad continuation chahiye thi).

```
for year in YEARS:
    start, end = year_range(window_key, year)
    ndbi_val, ndbi_n = ndbi_for_period(buffered_geom, start, end)
    lights_val, lights_n = lights_for_period(buffered_geom, start, end)
    villages.at[i, f"ndbi_{year}"] = ndbi_val
    villages.at[i, f"ndbi_n_{year}"] = ndbi_n
    villages.at[i, f"lights_{year}"] = lights_val
    villages.at[i, f"lights_n_{year}"] = lights_n
```

Purpose: Yahi is function ka dil hai — har gaon ke andar, ab ek *doosra* loop hai, teen saal ke liye. Har saal ke liye `year_range()` se date-range milta hai, fir wahi shared `ndbi_for_period()/lights_for_period()` functions call hote hain, aur result us saal ke specific column mein (`f"ndbi_{year}"`) likha jaata hai. Ye PDF 8 ke pattern se structurally alag hai — wahan ek row ke liye sirf 'before' aur 'after' do calls thi; yahan ek row ke liye teen alag saal ke liye calls hain, ek chhote loop mein. Syntax: Nested loop — bahar villages par, andar YEARS par. F-string column-naam, is baar assignment ke liye use ho rahe hain (`villages.at[i, f"ndbi_{year}"]`) — column-naam runtime par banta hai, saal ke hisaab se.

```
if (i + 1) % checkpoint_every == 0:
    villages.to_csv(out_path, index=False)
    print(f" {i + 1}/{len(villages)} villages processed...")

time.sleep(0.2)

villages.to_csv(out_path, index=False)
n_valid = villages.dropna(subset=[f"ndbi_{y}" for y in YEARS]).shape[0]
print(f"Done. {n_valid}/{len(villages)} villages have valid NDBI at all {len(YEARS)} time points.")
```

Purpose: Checkpoint, wait, final-save — wahi standard pattern. `n_valid` yahan check karta hai ki kitni villages ke paas *teeno* saal ka valid NDBI hai — list-comprehension se, teen column-naam ek saath banaakar, `dropna(subset=[...])` ko diye jaate hain. Syntax: `[f"ndbi_{y}" for y in YEARS]` — ek list-comprehension, teen column-naam ek list mein generate karta hai, dynamically. `dropna(subset==list)` — un sab columns mein se koi bhi missing ho toh row hata do.

__main__ Block

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Extract NDBI/VIIRS at 2021, 2023, 2025 for a multi-year trend.")
    parser.add_argument("--window", choices=["full_year", "summer"], required=True)
    args = parser.parse_args()

    init_ee()
    extract_window(args.window)
```

Purpose: PDF 8 ke `extract_satellite_data.py` jaisa hi `--window` flag, do options mein se ek chunne ke liye. Syntax: `argparse.ArgumentParser`, `.add_argument(...)`, `.parse_args()` — sab PDF 8 mein poori detail mein dekha gaya.

EXAMPLE — Sikkim Ka Archive-Backfill — Yahin Se Pakda Gaya

Is file ki sabse zaroori history-connection ye hai: ye file exist karti hai isliye kyunki iska comparison hi Entry 21-22 ke Sikkim-bug ko pakadne ka zariya bana. `extract_satellite_data.py` aur ye file, dono *identical* query chalte hain

summer-2021/summer-2025 ke liye (same collection, same cloud-mask, same buffer, same date-range) — lekin do alag dinon par extract hui. Result: 31 Sikkim treated villages, jo purani file mein dono periods ke liye 0 images dikha rahi thi (isliye NDBI null), is naye extraction mein 24 aur 31 scenes dikha rahi thi (real values, kuch bhi null nahi). Wajah — Sentinel-2 ka archive, purane dates ke liye bhi, samay ke saath naye scenes backfill karta rehta hai. Isi discovery ne is poore project ka sabse bada, single reversal (Entry 22) shuru kiya. Aur ek separate, positive finding bhi is file se aayi (Entry 13): teen-point trend fit karne se pata chala ki 2021-vs-2025 ka headline result asal mein 2021-se-2023 tak ek decline (mean change -0.023) ke baad, 2023-se-2025 tak ek strong, highly-significant increase ($p < 0.000001$) hai — ek steady trend nahi, ek dip-then-rise pattern, jo sirf do-point comparison se kabhi dikh nahi sakta tha.

File 2 — extract_pretreatment_baseline.py: 2019 Ka Ek Teesra, Pehle-Ka Point

Docstring — Ek Genuinely Naya Idea

```
"""
2019 baseline for the placebo test (Section 7.5) -- gives me a second
pre-treatment point (2019, 2021) for both groups, before VVP-I even existed.
Treated-only multiyear script can't do this since control has no 3rd point.

Needs live GEE, run on my machine.
"""
```

Purpose: Ye docstring ek subtle lekin bahut zaroori difference batata hai File 1 se. File 1 ka 2023 point, ek *treatment ke dauraan* ka extra point tha — teen point jinme se do (2021, 2025) already before/after hain aur beech ka (2023) treatment ke asar ko trace karta hai. Ye file bilkul alag kaam karta hai: 2019 ek *pehle-ka* point hai — VVP-I 2023 mein sanction hui thi, isliye 2019-se-2021 tak ka farak, treatment se pehle ka hai. Agar treated aur control villages, is treatment-se-pehle period mein bhi ek jaisa hi trend dikhaate hain, ye is study ke 'parallel pre-trends' assumption (jo har DiD design ki buniyaad hai) ke liye direct, real evidence hai — sirf ek theoretical assumption nahi. Docstring ye bhi explain karta hai ki File 1 (multi-year) ye kaam kyun nahi kar sakti: wo sirf treated villages ke liye hai, control ke paas teesra point hi nahi hai. Syntax: Multi-line docstring, jaisa poore project mein consistent style.

Imports Aur Path/Window Constants

```
S2_COLLECTION = "COPERNICUS/S2_SR_HARMONIZED"
VIIRS_COLLECTION = "NOAA/VIIRS/DNB/MONTHLY_V1/VCMSLCFG"
BUFFER_RADIUS_M = 500 # same buffer as everywhere else

VILLAGE_PATHS = {
    "treated": "data/processed/border_optics_master_villages.csv",
    "control": "data/processed/border_optics_control_villages.csv",
}
```

Purpose: Ye file, PDF 8 ki files se ek zaroori tareeke se alag hai — ye sirf treated ya sirf control ke liye nahi, *dono* ke liye chalti hai, ek hi script se. VILLAGE_PATHS ek dictionary hai jo 'treated' aur 'control' ko unki apni-apni village-list files se map karta hai. Ye is PDF ki teen files (ye, Dynamic World pretreatment, aur SAR) ka common pattern hai — ek hi script, ek --group flag se decide karti hai kaunsi list use karni hai. Syntax: Dictionary literal, do keys (string) ko do file-path strings se map karta hai.

```
WINDOWS = {
    "full_year": ("2019-01-01", "2020-01-01"),
    "summer": ("2019-06-01", "2019-10-01"),
}
```

```

OUT_PATHS = {
    ("treated", "full_year"):
    "data/processed/border_optics_treated_pretreatment_fullyear.csv",
    ("treated", "summer"):
    "data/processed/border_optics_treated_pretreatment_summer.csv",
    ("control", "full_year"):
    "data/processed/border_optics_control_pretreatment_fullyear.csv",
    ("control", "summer"):
    "data/processed/border_optics_control_pretreatment_summer.csv",
}

OUTCOME_COLS = ["ndbi_2019", "lights_2019", "ndbi_2019_image_count",
"lights_2019_image_count"]

```

Purpose: WINDOWS is baar *ek hi* date-range rakhta hai per window-type (comment khud kehta hai — 'single composite per village here, not before/after — 2019 IS the before for this placebo'). Matlab, is file mein koi 'before' aur 'after' nahi hai — sirf ek hi 2019 snapshot, jo khud placebo-test ka 'before' point banega (jiska 'after' point 2021 hai, jo already extract ho chuka hai PDF 8 ki files se). OUT_PATHS ek naya pattern dikhata hai — ek dictionary jiski *key khud ek tuple hai*: ("treated", "full_year"). Matlab, char combinations (2 groups × 2 windows) ke liye, char alag output-files. OUTCOME_COLS chaar column-naam, jinme koi 'before'/'after' suffix nahi — sirf _2019. Syntax: WINDOWS ek dictionary jiski values tuples hain (do dates ka jodaa, before/after ki jagah). OUT_PATHS ek dictionary jiski keys *tuples* hain — ("treated", "full_year") jaisa ek tuple, dictionary mein ek valid key ho sakta hai (Python mein tuples immutable hote hain, isliye hashable, isliye key-ban sakte hain — lists nahi ban sakti). Aage OUT_PATHS[(group, window_key)] se value nikaali jaayegi.

load_with_resume() — Do Groups, Do Alag Resume-Strategies, Ek Function

```
def _load_with_resume(group, out_path):
    # treated village_id is stable so resume by id; control list gets regenerated
    # each time so resume by coords instead (same fix as extract_control_satellite_data.py)
    villages = pd.read_csv(VILLAGE_PATHS[group])
    for col in OUTCOME_COLS:
        if col not in villages.columns:
            villages[col] = None

    if not os.path.exists(out_path):
        return villages
```

Purpose: Ye function ka naam khud (underscore-prefix, `_load_with_resume`) batata hai ki ye ek 'internal helper' hai — sirf isi file ke andar use hone ke liye, bahar se import nahi hota. Comment poori strategy ek line mein bata deta hai: treated ka `village_id` stable hai (kabhi nahi badalta, isliye usi se resume karo), control ka `village_id` har regeneration par badal jaata hai (isliye coordinate se resume karo — bilkul PDF 8 ke Entry 15 fix jaisa). Village-list padhna, missing columns bharna — usual. Agar output-file exist hi nahi karti, seedha fresh villages wapas kar do — resume karne ko kuch nahi hai. **Syntax:** Function-naam underscore se shuru hona — Python mein ye ek convention hai (enforced nahi, sirf ek signal) ki ye function 'private' hai, is module ke bahar use nahi hona chahiye. `os.path.exists(...)` check, negate kiya gaya (not) — 'agar file exist nahi karti.'

```
existing = pd.read_csv(out_path)

if group == "treated":
    if "village_id" in existing.columns:
        lookup = existing.set_index("village_id")[
            [c for c in OUTCOME_COLS if c in existing.columns]
        ].to_dict("index")
        for i, vid in zip(villages.index, villages["village_id"]):
            values = lookup.get(vid)
            if values:
                for col in OUTCOME_COLS:
                    if col in values:
                        villages.at[i, col] = values[col]
        print(f"Resuming from existing checkpoint: {out_path} ({len(existing)}) rows,
matched by village_id")
    return villages
```

Purpose: Ye treated-specific branch hai. Agar checkpoint file mein `village_id` column hai, ek lookup-dictionary banayi jaati hai — key `village_id`, value baaki outcome-columns ka dictionary. Fir har current village ke liye, uska `village_id` use karke, purani values wapas li jaati hain, agar mili toh. Ye PDF 8 ki coordinate-based resume se structurally alag hai, lekin same goal — kyunki treated ka `village_id` stable hai, yahan id-based lookup safe hai, coordinate ki zaroorat hi nahi. **Syntax:**

`existing.set_index("village_id")` — dataframe ka index badalna, default numeric position ki jagah `village_id` column ko index banana, taaki `.to_dict("index")` se ek id-to-row dictionary bane. `zip(villages.index, villages["village_id"])` — do sequences ko ek saath iterate karna, jodo mein.

```
current_coords = set(zip(villages["latitude"].round(6), villages["longitude"].round(6)))
existing_coords = (
    set(zip(existing["latitude"].round(6), existing["longitude"].round(6)))
    if {"latitude", "longitude"}.issubset(existing.columns) else set()
)
if not (existing_coords and existing_coords == current_coords):
    print(f"Checkpoint at {out_path} doesn't match current control list -- starting fresh.")
    return villages

lookup = existing.copy()
lookup["_coord_key"] = list(zip(lookup["latitude"].round(6),
lookup["longitude"].round(6)))
lookup_dict = lookup.set_index("_coord_key")[
    [c for c in OUTCOME_COLS if c in lookup.columns]
].to_dict("index")
villages_keys = list(zip(villages["latitude"].round(6), villages["longitude"].round(6)))
for i, key in zip(villages.index, villages_keys):
    values = lookup_dict.get(key)
    if values:
        for col in OUTCOME_COLS:
            if col in values:
                villages.at[i, col] = values[col]
print(f"Resuming from existing checkpoint: {out_path} ({len(existing)} rows, matched by
coordinate)")
return villages
```

Purpose: Agar upar wala `if group == "treated":` branch nahi chala (matlab group control hai), execution yahan pahunchta hai — bilkul wahi coordinate-based mismatch-check jo humne 'Shared Skeleton' section mein poori detail mein dekha. Yahi is function ki khoobsurati hai: ek hi function, andar ek `if` se, dono groups ke liye sahi resume-strategy chunta hai — `treated` ke liye stable-id-based, control ke liye coordinate-based. **Syntax:** Wahi coordinate-set-comparison syntax jo shared-skeleton mein dekha. Function ke do exit-points hain — ek `treated-branch` ke andar (`return villages` us `if` block ke andar hi), ek yahan control-path ke end mein.

extract() Aur __main__ — Dono Groups Ko Ek Command Se Chalana

```
def extract(group, window_key, checkpoint_every=10):
    start, end = WINDOWS[window_key]
    out_path = OUT_PATHS[(group, window_key)]
    villages = _load_with_resume(group, out_path)

    for i, row in villages.iterrows():
        if all(pd.isna(row.get(c)) for c in OUTCOME_COLS):
            continue

        point = ee.Geometry.Point([row["longitude"], row["latitude"]])
        buffered_geom = point.buffer(BUFFER_RADIUS_M)

        ndbi, ndbi_n = ndbi_for_period(buffered_geom, start, end)
        lights, lights_n = lights_for_period(buffered_geom, start, end)

        villages.at[i, "ndbi_2019"] = ndbi
        villages.at[i, "lights_2019"] = lights
        villages.at[i, "ndbi_2019_image_count"] = ndbi_n
        villages.at[i, "lights_2019_image_count"] = lights_n
```

Purpose: `OUT_PATHS[(group, window_key)]` — tuple-key se dictionary lookup, jaisa upar dekha. `_load_with_resume()` call karke sahi tareeke se loaded villages milti hain. Baaki loop bilkul PDF 8 jaisa hai — sirf ek hi date-range (before/after nahi), isliye `ndbi_for_period()/lights_for_period()` sirf ek-ek baar call hote hain, do baar nahi. **Syntax:** Tuple-key dictionary access — `OUT_PATHS[(group, window_key)]`, do variables se ek tuple banaakar seedha lookup mein use karna. Baaki loop-syntax familiar.

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Extract a 2019 pre-treatment baseline (NDBI, VIIRS lights) for a genuine parallel-pre-trends placebo test (Section 7.5)."
    )
    parser.add_argument("--group", choices=["treated", "control"], required=True)
    parser.add_argument("--window", choices=["full_year", "summer"], required=True)
    args = parser.parse_args()

    init_ee()
    extract(args.group, args.window)
```

Purpose: Do flags is baar — `--group` aur `--window` — matlab, ye script char alag baar chalani padti hai (docstring mein bhi ye char commands diye gaye hain), har group-window combination ke liye ek baar. **Syntax:** Do `.add_argument(...)` calls, dono `required=True`. `args.group`, `args.window` dono attributes access karna.

EXAMPLE — Entry 30 → 39 → 42 — Ek Result Jo Teen Baar Report Hua, Har Baar Thoda Sahi Hokaar

Is file ki run-history ek achha example hai is project ke apne discipline ka. Entry 28 mein ye script sirf likhi gayi thi. Entry 30 mein pehli baar chalayi gayi — 732-village control list par — teen combinations (full-year NDBI, full-year lights, summer lights) saaf clean nikle (koi significant pre-treatment farak nahi), lekin summer NDBI ek borderline exception nikla: coefficient $+0.01287$, $p=0.06429$ — 0.05 se thoda upar, lekin close, aur raw Mann-Whitney comparison mein sharply significant ($p=0.00001$). Entry 30 ne isse turant 'PASS' nahi maana sirf isliye ki script ka apna auto-verdict function bare $p>0.05$ check karta hai — is borderline case ko explicitly flag kiya gaya. Entry 39 mein pata chala ki ye poora extraction abhi bhi purani 732-village control list par tha (721-village fix ke baad bhi) — is gap ko chhupaya nahi gaya, balki har jagah explicitly 'vintage caveat' likha gaya jahan ye numbers quote hote hain. Aakhir mein, Entry 42 mein isi script ko --group control ke saath dobara chalaya gaya, is baar corrected 721-village list par — result: summer NDBI abhi bhi borderline hai ($+0.01173$, $p=0.07421$), baaki teeno abhi bhi clean. Ek achhi baat: ye borderline pre-period effect (treated villages summer mein thoda upar trend kar rahe the, treatment se pehle hi) *ulti* direction mein hai us already-null summer NDBI DiD result (-0.00275) se — matlab, ye pre-existing trend, us result ko explain nahi kar sakta jo already null hai.

File 3 — extract_dw_pretreatment_baseline.py: Dynamic World Ka Apna Placebo

Docstring — Ek Dusre Proxy Ke Liye Wahi Sawaal

```
"""
2019 Dynamic World baseline -- the DW analogue of extract_pretreatment_baseline.py.
Suggestion from an external review: since Dynamic World shows a small but significant
control-group effect (Section 6, triangulation) while NDBI doesn't, a genuine
placebo test on DW itself (not just on NDBI/lights) is worth having before
trusting that DW gap as real rather than a pre-existing trend.
"""
```

Purpose: Ye file, File 2 ka 'DW version' hai — bilkul wahi placebo-test idea, lekin NDBI/lights ki jagah Dynamic World ke 'built' probability band par. Docstring khud reasoning deta hai: PDF 9 aur PDF 8 mein humne dekha ki Dynamic World, NDBI ke ulat, treated-control DiD mein ek chhota lekin significant effect dikhaata hai. Isse pehle ki us effect ko 'real' maan liya jaaye, ek zaroori sawaal poochna chahiye: kya treated aur control villages, treatment se *pehle* hi is direction mein alag the Dynamic World ke hisaab se? Agar haan, toh 2025 ka gap ek naya effect nahi, ek purana, pehle-se-chal- raha trend ho sakta hai. Syntax: Multi-line docstring, is baar external-review-suggestion ko credit deta hai (bina tool ka naam liye, jaisa is project mein baad mein convention ban gaya).

Constants — File 2 Se Milta-Julta, Lekin DW Ke Liye

```
DW_COLLECTION = "GOOGLE/DYNAMICWORLD/V1"
BUFFER_RADIUS_M = 500 # same buffer as extract_dynamicworld_built.py / everywhere
else

VILLAGE_PATHS = {
    "treated": "data/processed/border_optics_master_villages.csv",
    "control": "data/processed/border_optics_control_villages.csv",
}

WINDOWS = {
    "full_year": ("2019-01-01", "2020-01-01"),
    "summer": ("2019-06-01", "2019-10-01"),
}

OUT_PATHS = {
    ("treated", "full_year"):
        "data/processed/border_optics_treated_dw_pretreatment_fullyear.csv",
    ("treated", "summer"):
        "data/processed/border_optics_treated_dw_pretreatment_summer.csv",
    ("control", "full_year"):
        "data/processed/border_optics_control_dw_pretreatment_fullyear.csv",
```

```

("control", "summer"):
    "data/processed/border_optics_control_dw_pretreatment_summer.csv",
}

```

```

OUTCOME_COLS = ["built_2019", "built_2019_image_count"]

```

Purpose: Structure bilkul File 2 jaisi — VILLAGE_PATHS, WINDOWS (same 2019 dates), OUT_PATHS (tuple-key pattern, same). Do farak: DW_COLLECTION Sentinel-2 ki jagah Google Dynamic World ka collection-path hai, aur OUTCOME_COLS sirf do columns rakhta hai (NDBI/lights ke char ki jagah) — kyunki Dynamic World ek hi quantity deta hai, 'built' probability. Syntax: Same dictionary patterns jo File 2 mein dekhe — sirf values badli hain, structure wahi.

built_prob_for_period() — Dynamic World Se 'Built' Probability Nikaalna

```

def built_prob_for_period(buffered_geom, start, end):
    collection = (
        ee.ImageCollection(DW_COLLECTION)
        .filterBounds(buffered_geom)
        .filterDate(start, end)
        .select("built")
    )
    count = collection.size().getInfo()
    if count == 0:
        return None, 0
    composite = collection.median()
    stats = composite.reduceRegion(
        reducer=ee.Reducer.mean(), geometry=buffered_geom, scale=10, maxPixels=1e9,
    ).getInfo()
    return stats.get("built"), count

```

Purpose: Ye function, is PDF mein *pehli baar* appear ho raha hai (File 5, extract_dynamicworld_built.py mein bhi identical hai — wahan reference karenge, na ki dobara). Google Dynamic World ek machine-learning classifier hai jo har Sentinel-2 pixel ko categories mein classify karta hai (water, trees, built-area, aur waise). .select("built") sirf 'built' category ka probability-band nikaalta hai — har pixel ke liye ek number 0 se 1 ke beech, 'kitni confidence hai ki ye pixel built-up hai.' Baaki structure NDBI jaisa hi hai — filter, count-check, median composite, reduceRegion se area-average, 10-meter scale par (Dynamic World bhi 10m resolution par hai, Sentinel-2 jaisa). Syntax: .select("built") — ek specific band chunna, jaisa .select("QA60") ya .select("avg_rad") pehle dekha. Baaki syntax (.median(), reduceRegion(...)) hu-ba-hu wahi jo NDBI ke liye dekha, sirf formula ki jagah seedha ek existing band ka average liya ja raha hai.

EXAMPLE — GHSL Ko Jaan-Bujh Kar Chhoda Gaya

File 5 (`extract_dynamicworld_built.py`) ka docstring ek zaroori design-decision explain karta hai jo yahan bhi apply hoti hai: ek doosra option, GHSL (Global Human Settlement Layer), jaan-bujh kar skip kiya gaya, kyunki uske future epochs 'modelled, not observed' hain — matlab, GHSL ka future data, asli satellite observations se nahi, balki ek model se predict kiya gaya hota hai. Agar use use karte, 'independent cross-check' hone ka daava hi galat hota — tu asal mein satellite se compare nahi kar rahi hoti, ek prediction-model se kar rahi hoti. Dynamic World, ismein contrast, asli Sentinel-2 imagery se, asli-time classify hota hai — isliye ek genuine (agar sensor-independent nahi, toh bhi algorithm-independent) cross-check hai.

_load_with_resume() Aur extract() — File 2 Se Reference

Ye function, aur uske baad ka `extract()`, File 2 ke `_load_with_resume()/extract()` se — `treated-by-village_id`, `control-by-coordinate`, dono logic samet — `character-by-character` identical hain, sirf column-naam `ndbi_2019/lights_2019` ki jagah `built_2019` hai. Isliye yahan dobara poori derivation nahi dohrayenge — jo upar File 2 mein dekha, wahi yahan bhi apply hota hai, line-by-line.

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Extract a 2019 Dynamic World 'built' baseline for a DW-specific
parallel-pre-trends placebo test."
    )
    parser.add_argument("--group", choices=["treated", "control"], required=True)
    parser.add_argument("--window", choices=["full_year", "summer"], required=True)
    args = parser.parse_args()

    init_ee()
    extract(args.group, args.window)
```

Purpose: Same do-flag pattern jo File 2 mein dekha — `--group` aur `--window`, char baar chalaane ke liye. Syntax: Identical `argparse` setup jo File 2 mein poori detail mein dekha.

EXAMPLE — Entry 32 → 36 → 37 → 43 — Sabse Lambi Wait Karne Wala Script

Is file ki kahani, is poore PDF mein sabse zyada Development-Log-entries cross karti hai. Entry 32 mein sirf idea aur intent likhi gayi. Entry 36 mein actual script likhi gayi, ek external review ke suggestion ke jawaab mein, lekin turant chalayi nahi ja saki — live GEE access chahiye thi. Entry 37 mein, jab control-list 732 se 721 ho gayi, is file mein ek zaroori fix hui: control-side ka join originally `village_id` se ho raha tha (jo control ke liye unstable hai) — is bug ne kuch corrupt nahi kiya tha abhi tak (kyunki extraction hui hi nahi thi), lekin agar chalaayi jaati bina fix ke, galat villages jud jaate. Fix hua: coordinate-based join, matching warning ke saath. Aakhir mein Entry 43 mein, poore chaar combinations (`treated/control × full-year/summer`) chalaye gaye, 721-village corrected list par. Result: pura clean. Full-year: $n=962$, coefficient $+0.00009$, $p=0.93406$. Summer: $n=652$, coefficient $+0.00207$, $p=0.17869$. Matlab, Dynamic World mein treated aur control villages ke beech, treatment se pehle, koi pre-existing divergence *nahi* thi — kisi bhi window mein. Ye ek genuine, naya finding hai: ye PDF 8/9 ke Dynamic World-vs-NDBI disagreement ke do possible explanations mein se ek ko khaarij karta hai — 'DW ka effect sirf ek purana, pehle-se-chal- raha trend continue ho raha hai' — lekin doosri possibility (ki DW ek real, chhota effect pakad raha hai jo NDBI/SAR 500m buffer par miss kar rahe hain, ya ki ye DW ke classifier ka apna specific false-positive hai) abhi bhi open hai.

File 4 — extract_sar_backscatter.py: Ek Bilkul Alag Satellite

Docstring — Kyun Radar Genuinely Different Hai

```
"""
Third-ish triangulation check, this time actually sensor-independent (Sentinel-1
radar, not just a different algorithm on the same S2 imagery like Dynamic World).
Also cloud-proof, which matters given the archive-timing bugs I've hit before.
Reporting raw VV/VH, not a derived index -- no standard "SAR built-up index" exists.
"""
```

Purpose: Ye docstring ek zaroori distinction khinchta hai jo poore is PDF mein important hai: Dynamic World 'algorithm-independent' hai (alag machine-learning model), lekin 'sensor-independent' nahi (wahi Sentinel-2 imagery use karta hai jo NDBI bhi use karta hai). SAR (Synthetic Aperture Radar, Sentinel-1 satellite) *dono* hai — bilkul alag satellite, bilkul alag physics (radar waves, optical light nahi). Ye is study ka sabse strong, genuinely independent cross-check hai. Ek aur fayda docstring mein hai: radar clouds ke through dekh sakta hai (radar waves clouds se block nahi hote jaise visible light hoti hai) — isliye is study ke apne, baar-baar-hue cloud-archive-timing bugs (Entries 21-22, buffer-radius same-day gap) is file ke liye structurally hi possible nahi hain. Aakhri line ek honest limitation batata hai — koi standard 'SAR built-up index' exist nahi karta (NDBI jaisa formula), isliye ye file raw VV aur VH values report karta hai, ek derived index nahi. Syntax: Multi-line docstring.

Constants — ORBIT_PASS, Ek Naya Concept

```
S1_COLLECTION = "COPERNICUS/S1_GRD"
BUFFER_RADIUS_M = 500 # same as extract_satellite_data.py's primary buffer
ORBIT_PASS = "DESCENDING" # if a run below prints 0 scenes, switch to 'ASCENDING'
and re-run

VILLAGE_PATHS = {
    "treated": "data/processed/border_optics_master_villages.csv",
    "control": "data/processed/border_optics_control_villages.csv",
}

WINDOWS = {
    "full_year": {"before": ("2021-01-01", "2022-01-01"), "after": ("2025-01-01",
"2026-01-01")},
    "summer": {"before": ("2021-06-01", "2021-10-01"), "after": ("2025-06-01", "2025-10-01")},
}

OUT_PATHS = {
    ("treated", "full_year"): "data/processed/border_optics_treated_sar_fullyear.csv",
    ("treated", "summer"): "data/processed/border_optics_treated_sar_summer.csv",
    ("control", "full_year"): "data/processed/border_optics_control_sar_fullyear.csv",
}
```

```
("control", "summer"): "data/processed/border_optics_control_sar_summer.csv",  
}
```

Purpose: Wahi VILLAGE_PATHS/tuple-key OUT_PATHS pattern. WINDOWS is baar File 2/3 se alag hai — inme before/after dono hain (kyunki ye 2019-placebo nahi, ye 2021-vs-2025 ka wahi primary triangulation-check hai jo PDF 8 mein NDBI ke liye tha). Naya, is file ke liye specific: ORBIT_PASS = "DESCENDING" — radar satellites, Earth ke around, ek specific direction mein orbit karte hain jab wo kisi jagah ke upar se guzarte hain (ascending ya descending) — aur ek hi jagah ka radar-image, ascending aur descending pass mein, geometrically thoda alag hota hai (radar ek angle se dekhta hai, camera ki tarah seedha neeche nahi). Isliye dono ko mix karna noise add karta — is script mein fix ek orbit-pass chuna gaya hai, aur comment khud ek fallback-instruction deta hai agar wo pass kisi area ko cover na kare. Syntax: Ek naya string constant, comment ke saath jo runtime-troubleshooting instruction deta hai. Baaki dictionaries familiar patterns.

sar_for_period() — VV Aur VH, Dono Polarizations

```
def sar_for_period(buffered_geom, start, end):
    # fixed orbit pass -- mixing ascending/descending adds noise, geometry isn't
    comparable
    collection = (
        ee.ImageCollection(S1_COLLECTION)
        .filterBounds(buffered_geom)
        .filterDate(start, end)
        .filter(ee.Filter.eq("instrumentMode", "IW"))
        .filter(ee.Filter.eq("orbitProperties_pass", ORBIT_PASS))
        .filter(ee.Filter.listContains("transmitterReceiverPolarisation", "VV"))
        .filter(ee.Filter.listContains("transmitterReceiverPolarisation", "VH"))
    )
    count = collection.size().getInfo()
    if count == 0:
        return None, None, 0
```

Purpose: Ye function NDBI/lights ke functions se zyada filters use karta hai, kyunki Sentinel-1 collection mein images alag-alag configurations mein aati hain aur unhe genuinely consistent banane ke liye char filters chahiye. `instrumentMode == "IW"` — 'Interferometric Wide swath,' Sentinel-1 ka sabse common land-imaging mode (doosre modes different resolution/coverage dete hain, unhe exclude karna zaroori hai consistency ke liye). `orbitProperties_pass == ORBIT_PASS` — sirf descending-pass images (jaisa upar constant mein set hai). Do `listContains` filters — dono VV aur VH polarizations chahiye is image mein (radar, alag-alag 'polarization' combinations mein signal bhejta aur receive karta hai — VV aur VH do alag combinations hain, jo alag surface-properties ke liye sensitive hote hain). Zero-count par null-safe return, is baar *teen* values ke saath (VV, VH, count) — do value-columns hone ki wajah se. Syntax: Char alag `.filter(ee.Filter...)` calls, chain mein. `ee.Filter.eq(field, value)` — exact match filter. `ee.Filter.listContains(field, value)` — check karta hai ki ek list-type field mein wo specific value hai ya nahi (yahan, polarization-list mein VV/VH dono hain ya nahi). `return None, None, 0` — teen values, do None ek saath (VV aur VH dono unavailable).

```
composite = collection.select(["VV", "VH"]).median()
stats = composite.reduceRegion(
    reducer=ee.Reducer.mean(),
    geometry=buffered_geom,
    scale=20, # IW footprint is coarser than 10m
    maxPixels=1e9,
).getInfo()
return stats.get("VV"), stats.get("VH"), count
```

Purpose: `.select(["VV", "VH"])` — is baar ek saath *do* bands select kiye ja rahe hain (list mein), taaki ek hi `reduceRegion` call dono ka average de de, alag-alag calls ki zaroorat nahi. Median composite, wahi pattern. `scale=20` — NDBI ke 10-meter se alag, kyunki comment khud kehta hai: Sentinel-1 ka IW mode footprint, Sentinel-2 se coarser (kam

detailed) hai, isliye 20-meter scale zyada appropriate hai, extra precision jo data mein hai hi nahi use fake create karne se better. Syntax: `.select([list])` — ek saath multiple bands chunna, single-band `.select("string")` se alag. `stats.get("VV")`, `stats.get("VH")`, `count` — teen values return, comma se separated, tuple ban jaata hai.

extract() — Mismatch-Check Ka Comment Khud Bug Ki History Batata Hai

```
def extract(group, window_key, checkpoint_every=10):
    cfg = WINDOWS[window_key]
    before_start, before_end = cfg["before"]
    after_start, after_end = cfg["after"]
    out_path = OUT_PATHS[(group, window_key)]

    outcome_cols = [
        "vv_before", "vh_before", "vv_after", "vh_after",
        "sar_before_image_count", "sar_after_image_count",
    ]

    # village list is the source of truth ... this script used to just load
    # the checkpoint outright with no mismatch check, which is what let a
    # post-district-verification-rerun run silently reuse the old 732-village
    # control checkpoint instead of the corrected 721-village list.
    villages = pd.read_csv(VILLAGE_PATHS[group])
```

Purpose: Config unpacking aur outcome_cols definition, familiar patterns. Sabse zaroori hissa yahan comment hai — ye is file ka apna, permanent record hai ki ismein pehle ye coordinate-mismatch-check tha hi nahi, aur uski wajah se exactly kya hua (Entry 37 ki 'silently reuse the old 732-village checkpoint' kahani, jo humne shared-skeleton section mein dekhi). Ab ye check yahan present hai, neech wahi block jo 'Shared Skeleton' mein poori detail mein dekha gaya. Syntax: Tuple-unpacking do baar (before, after). List literal, chaar+ column-naam.

Neeche ka poora coordinate-mismatch-check block (if os.path.exists(out_path): ...), aur uske baad ka main loop (point/buffer banana, sar_for_period() ko before/after ke liye call karna, results save karna, checkpoint/sleep, final summary) — sab is PDF ke 'Shared Skeleton' section mein poori detail mein dekha ja chuka hai. Yahan sirf itna naya hai ki columns VV/VH ke liye hain, NDBI/lights ke liye nahi — baaki structure hu-ba-hu wahi.

EXAMPLE — Entry 26 — Ek Checkpoint Jo Exist Karti Thi, Lekin Poori Nahi Thi

Is file ki pehli badi kahani, resume-mismatch-check se bhi pehle ki hai. Jab SAR pehli baar chalayi gayi (Entry 26), control-summer file mein sirf 50 of 732 villages ke paas valid VV/VH values thi — aur, zaroori baat, baaki 682 ke image-count columns bhi *null* the (zero nahi) — matlab, extraction-loop un rows tak pahuncha hi nahi tha, ruk gaya tha kahin beech mein. Iska resume-logic khud buggy nahi tha, lekin ek checkpoint-file ka exist karna, iska matlab nahi hota ki uske peeche ka run poora hua. Ye tabhi pakda gaya jab actual analysis-script (triangulation_analysis.py) ke against directly count kiya gaya, file ke exist karne par bharosa karne ki jagah. Fix simple tha — script ko dobara chalaya gaya, aur uske apne resume-logic ne sahi jagah se, 50 done rows skip karke, baaki 682 process kiye, 732/732 tak. Interesting side-note: us adhoori 50-village file par manual-OLS se check karne par VV/VH dono

spuriously significant lag rahe the ($p=0.014$, $p=0.0001$) — sirf isliye kyunki wo 50 villages ek random sample nahi thi, extraction-order ki 'first 50' thi. Poora extraction hone ke baad, dono null ($p=0.717$, $p=0.953$).

Metric	Window	H1 (treated-only)	H4 (control DiD, 721-list)
SAR VV (dB)	Full-year	mean change -0.099, p=1.000	coef -0.008, p=0.791
SAR VV (dB)	Summer	mean change -0.177, p=1.000	coef +0.012, p=0.717
SAR VH (dB)	Full-year	mean change -0.099, p=1.000	coef -0.008, p=0.774
SAR VH (dB)	Summer	mean change -0.154, p=1.000	coef -0.002, p=0.953

SAR — is study ka sabse zyada genuinely sensor-independent check — dono bands, dono windows, har specification mein clean null hai. Entry 26 ke apne shabdon mein: 'a completely independent sensor was pointed at the same villages and found nothing either.' Ye is study ke headline null ke liye is poori PDF ki sabse mazboot, sabse confirmatory evidence hai.

File 5 — extract_dynamicworld_built.py: Wahi Sawaal, Alag Algorithm Se

Docstring

```
"""
```

```
Cross-check for H1/H4: pulls Dynamic World's "built" probability (different algorithm than my NDBI, though same S2 imagery underneath) for the same villages/buffers/windows. Skipped GHSL on purpose -- its future epochs are modelled not observed, so it'd be a fake "independent" check.
```

```
"""
```

Purpose: Ye vahi GHSL-explanation jo humne File 3 mein EXAMPLE ke roop mein dekha, ab iske apne original source docstring mein. Ye file, File 3 ki 2019-baseline version se pehle bani thi — ye hai wo file jo 2021-vs-2025 (primary comparison) ke liye Dynamic World data khinchti hai, File 3 sirf uska 2019-placebo extension hai. Syntax: Multi-line docstring.

DW_COLLECTION, BUFFER_RADIUS_M, VILLAGE_PATHS, aur — sabse zaroori — built_prob_for_period() function, ye sab File 3 mein bilkul identical dekhe ja chuke hain (File 3 hi inhe reuse karta hai). WINDOWS yahan before/after dono rakhta hai (2021/2025 primary comparison ke liye, File 3 ke single-2019-snapshot se alag) — bilkul File 4 (SAR) ke WINDOWS jaisa structure.

```
WINDOWS = {
    "full_year": {"before": ("2021-01-01", "2022-01-01"), "after": ("2025-01-01",
"2026-01-01")},
    "summer": {"before": ("2021-06-01", "2021-10-01"), "after": ("2025-06-01", "2025-10-01")},
}

outcome_cols = ["built_before", "built_after", "built_before_image_count",
"built_after_image_count"]
```

Purpose: Char outcome-columns, before/after pattern (File 4/SAR jaisa), built_before/built_after naam ke saath — File 3 ke built_2019 (sirf ek value) se alag, kyunki yahan genuine before/after difference chahiye. Syntax: List literal, chaar column-naam.

extract() function ka poora structure — same coordinate-mismatch-check comment (word-for-word File 4 jaisa hi, Entry 37 ki reference ke saath), same main loop (is baar built_prob_for_period() ko before aur after ke liye call karna) — sab 'Shared Skeleton' section mein poori detail mein dekha ja chuka hai. Yahan koi genuinely naya code-pattern nahi hai jo pehle na dekha ho.

Window	H1 (treated-only)	H4 (control DiD)
Full-year	n=249, mean +0.00673, p<0.000001	coef +0.00310, p=0.01168 (732-list)
Summer	n=169, mean +0.00711, p<0.000001	coef +0.00736, p=0.00133 (732-list)

Ye numbers Entry 26 ke pehle-run se hain (732-village list par). 721-village list par re-run karne ke baad (Entry 37, 39), ye significant reh jaate hain lekin numbers thoda badalte hain (0.0159/0.0006 — jo PDF ki closing synthesis mein dekhenge). Dono windows mein, Dynamic World treated villages mein control ke muqable ek chhota (+0.003 se +0.007), lekin real aur significant increase dikhaata hai — jabki NDBI aur SAR dono null hain. Ye is study ka sabse bada, abhi bhi khula sawaal hai.

EXAMPLE — Char Findings, Ek Kahani — Entry 27 Ka Investigation

Jab ye disagreement pehli baar mila (Entry 26), sawaal turant nahi tha 'kaunsa sahi hai' — sawaal tha 'kya ye ek genuine signal hai ya koi artifact.' Entry 27 ne char specific checks kiye. Ek: outlier-drop karne se result barely badla (+0.00673 se +0.00621) — matlab, ye kuch handful villages ka artifact nahi, broad-based hai. Do: ek genuine, unexplained asymmetry mili — treated villages mein, jo pehle se hi zyada built-up thi, unme hi zyada increase dikha ($\rho=+0.272$), jabki control villages mein ulta pattern tha ($\rho=-0.229$ summer mein) — do explanations possible hain (real 'existing settlements densify preferentially' pattern, ya measurement-sensitivity artifact), dono open chhodi gayi. Teen: ek structural echo mila Entries 21-22 ke Sikkim-bug ka — saari 31 Sikkim treated villages summer window mein null hain, full-year mein sab valid — bilkul wahi shape jo NDBI mein pehle mila tha, is baar Dynamic World ke apne Sentinel-2 pull mein, resolve nahi kiya gaya (genuine monsoon-cloud ho sakta hai, ya archive-backfill, confirm nahi kiya gaya). Char: image-count correlation, treated aur control mein *opposite direction* mein — dono weak lekin real. Koi bhi ek cheez poora disagreement explain nahi karti; ye ek genuine, multi-layered, abhi-bhi-khula complication hai.

File 6 — extract_building_footprints.py: Satellite Proxy Se Utar Kar, Asli Buildings Tak

Docstring — Ek Scope-Correction Upfront

```
"""
Building-footprint check (Section 7.2). Open Buildings is single-vintage, no
2021 vs 2025 to diff like NDBI, so this is just a current-state validation
check -- mainly for the Walong/Kaho/Musai ground-truth question.
"""
```

Purpose: Ye docstring ek zaroori honesty upfront rakhta hai — jab ye file plan ki gayi thi, socha gaya tha ki ye bhi before/after comparison karegi, baaki files jaisa. Lekin check karne par pata chala (Entry 29): Google Open Buildings — jo ye file use karti hai — ek *single-vintage* dataset hai, matlab iska sirf ek hi 'snapshot' hai, 2021 ka alag aur 2025 ka alag version exist hi nahi karta. Isliye ye file, apne design mein hi, ek current-state check ban gayi — sirf abhi kitni buildings hain, unhe count karna, before/after nahi. Docstring khud kehta hai ye scope-narrowing pehle se hi jaani-boojhi decision thi, baad mein discovered limitation nahi. Syntax: Multi-line docstring, honest scope-statement.

Constants

```
OPEN_BUILDINGS_COLLECTION = "GOOGLE/Research/open-buildings/v3/polygons"
BUFFER_RADIUS_M = 500 # same buffer as everywhere else
CONFIDENCE_THRESHOLD = 0.75 # Open Buildings' default "high confidence" cutoff

VILLAGE_PATHS = {"treated": "data/processed/border_optics_master_villages.csv",
                  "control": "data/processed/border_optics_control_villages.csv"}
OUT_PATHS = {"treated":
              "data/processed/border_optics_treated_building_footprints.csv",
              "control": "data/processed/border_optics_control_building_footprints.csv"}
OUTCOME_COLS = ["building_count", "building_total_area_m2",
                 "building_mean_confidence"]
```

Purpose: OPEN_BUILDINGS_COLLECTION — Google ke Open Buildings dataset ka path, jo satellite imagery se machine-learning ke through detect ki gayi asli building-outlines (polygons) rakhta hai. CONFIDENCE_THRESHOLD = 0.75 — har detected building ke saath ek confidence-score aata hai (kitna sure hai model ki ye asal mein ek building hai); 0.75 Open Buildings ka apna, standard 'high confidence' cutoff hai, isliye low-confidence false-positives (jaise chhoti chattaanein ya galat detections) exclude ho jaate hain. VILLAGE_PATHS/OUT_PATHS is baar tuple-key nahi, simple group-key dictionaries hain (kyunki koi window-dimension nahi hai — single snapshot hai, do nahi). OUTCOME_COLS teen naye columns — count, total-area, mean-confidence. Syntax: String/float constants. Do simple (non-tuple-key) dictionaries — VILLAGE_PATHS[group] aur OUT_PATHS[group] seedha group-naam

se access honge.

buildings_for_buffer() — Polygons Ginna Aur Unka Area Jodna

```
def buildings_for_buffer(buffered_geom):
    # NOTE: filterBounds selects any building polygon that INTERSECTS the buffer, but
    # `area_in_meters` is that polygon's precomputed full area, not clipped to the buffer.
    # ... `building_total_area_m2` should be treated as an upper bound, not
    # an exact buffer-clipped area, until this is fixed with a `.intersection()`/clip step.
    fc = (
        ee.FeatureCollection(OPEN_BUILDINGS_COLLECTION)
        .filterBounds(buffered_geom)
        .filter(ee.Filter.gte("confidence", CONFIDENCE_THRESHOLD))
    )
    count = fc.size().getInfo()
    if count == 0:
        return 0, 0.0, None
```

Purpose: Ye is file ka core function hai, aur iska comment ek honest, self-flagged limitation record karta hai — code likhte waqt hi. Pehle ek FeatureCollection banti hai (Earth Engine mein, points/lines/polygons jaisi 'vector' geometries ka collection, images ki jagah) — Open Buildings se, buffer ko intersect karne wali saari building-polygons, sirf 0.75+ confidence ki. Comment explain karta hai ek subtlety: filterBounds koi bhi polygon include karta hai jo buffer ko *choo* bhi rahi ho (poori tarah andar na ho), lekin uska 'area' field poori polygon ka area hota hai, sirf buffer ke andar wala hissa nahi. Matlab, agar ek badi building buffer ki boundary par hai, uska poora area count ho jaayega, sirf jo hissa andar hai wo nahi — isliye total-area ek 'upper bound' hai, exact nahi. Ye building_count ko affect nahi karta (wo sirf polygons ginta hai), sirf building_total_area_m2 ko. Zero-buildings case mein, 0, 0.0, None return hota hai (count 0, area 0.0, confidence-average None kyunki average-of-nothing undefined hai). Syntax: ee.FeatureCollection(path) — vector-data collection banane ka tareeka, ee.ImageCollection se alag data-type. .filter(ee.Filter.gte("confidence", threshold)) — gte = 'greater than or equal to,' ek numeric field-filter. return 0, 0.0, None — teen alag default values, mixed types (int, float, None).

```
stats = fc.aggregate_stats("area_in_meters").getInfo()
conf_mean = fc.aggregate_mean("confidence").getInfo()
total_area = stats.get("sum", 0.0) if stats else 0.0
return count, total_area, conf_mean
```

Purpose: aggregate_stats() ek FeatureCollection ke ek numeric field par, summary-statistics (sum, mean, min, max, etc.) ek saath deta hai — yahan sirf "sum" chahiye, jo total area hai saari matching buildings ka. aggregate_mean() seedha average deta hai, ek field ka (confidence scores ka average, is buffer ki saari buildings ka). stats.get("sum", 0.0) — safe default ke saath dictionary-access, agar "sum" key na mile toh 0.0. Syntax: .aggregate_stats(field)/.aggregate_mean(field) — Earth Engine ke FeatureCollection-specific aggregation functions, ImageCollection ke reduceRegion se alag (kyunki data-type khud alag hai). dict.get(key, default) — do-argument form,

jisme default value bhi diya jaata hai agar key na mile.

extract() function ka structure — coordinate-mismatch-check comment (is baar comment explicitly kehta hai 'same bug found and fixed in extract_dynamicworld_built.py / extract_sar_backscatter.py after a district-verification rerun' — matlab, ye file *pehle se hi* fix ke saath likhi gayi thi, kyunki ye baad mein likhi gayi jab tak Entry 37 ka bug pattern already pata chal chuka tha), fir main loop — sab 'Shared Skeleton' mein dekha ja chuka hai. Yahan naya sirf ye hai ki iske paas koi --window flag nahi hai, kyunki single snapshot hai, before/after nahi.

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Extract Open Buildings footprint count/area per village buffer (single
current snapshot; Section 7.2)."
    )
    parser.add_argument("--group", choices=["treated", "control"], required=True)
    args = parser.parse_args()

    init_ee()
    extract(args.group)
```

Purpose: Sirf ek flag, --group — File 2/3/4/5 ke do-flag pattern se simpler, kyunki koi window-dimension nahi hai. Syntax: Single .add_argument(...) call, baaki familiar.

EXAMPLE — Walong, Kaho, Musai — Jab Satellite Data Asli Zameen Se Mila

Is file ki sabse important history, extraction se pehle ki hai. Entry 28 mein, ek official government tour (Arunachal ke Deputy CM, Anjaw district, October-November 2023) ki news-coverage mein 15 specific, named, dated projects mile. In 15 mein se teen villages — Walong, Kaho, Musai — is study ke 258-village sample mein already the. Ye is poore study ka *pehla genuine ground-truth positive control* tha — matlab, wo cases jahan 'kuch naya bana' ye independently, news se confirm ho, na ki sirf satellite se guess kiya gaya ho. In teeno villages ka har proxy mein before-after check kiya gaya (naya extraction ki zaroorat nahi thi — jo data pehle se tha, usi se): VIIRS night-lights ne dono confirmed-construction villages (Walong, Kaho) mein bada increase pakda (+0.909, ~doubling; aur +0.420, +82%). NDBI ne *ulti* direction mein move kiya teeno mein — despite confirmed new construction. Dynamic World ne chhota, sahi-direction change dikhaya teeno mein. SAR ne kuch specific nahi dikhaya, apne pooled null ke consistent. Ye NDBI ke apne pooled-null result ke liye ek genuine, sourced explanation deta hai: confirmed projects (solar streetlights, ek hostel, ek basketball court) point-scale hain, aur 500m-radius reflectance-average mein terrain/vegetation/existing-built-surface ke against, ek rounding error jaisi hain.

Proxy	Building-Footprint Rho (vs. 2025 level)
NDBI	0.399 (p<0.00001)
Night-lights	0.478 (p<0.00001)
Dynamic World "built"	0.808 (p<0.00001)

Entry 30 mein, is extraction ke actual chalne ke baad, ek doosra, alag finding mila — ye level-correlation hai, change nahi, isliye Section 4.10 ke disagreement ko resolve nahi karta, lekin ek zaroori context deta hai: Dynamic World ka correlation asli building-footprint-counts ke saath, NDBI ke roughly double hai. Matlab, Dynamic World, is terrain mein, ground-truth built-structure ko NDBI se kaafi behtar track karta hai — jo bataata hai ki Dynamic World ka Section 4.10 wala disagreement, sirf noise samajh kar khaarij nahi karna chahiye. Walong (524 buildings), Kaho (78), Musai (108) — sample median 34 ke against — confirm karte hain ki teeno genuinely real, moderate-to-large settlements hain, chhoti-si hamlets nahi jahan ek chhota addition percentage mein bada dikhta.

File 7 — `extract_buffer_sensitivity_data.py`: Zoom In, Zoom Out

Is file ka top hissa dhyaan se dekho — koi module docstring hi nahi hai (jaisa har doosri file mein tha), sirf do khaali lines fir seedha imports. Ye is project ki pehli file hai is PDF mein jismein top-level documentation missing hai — chhota detail, lekin worth noticing, kyunki baaki har file ismein consistent rahi hai.

Constants Aur Shared Functions

```
VILLAGES_PATH = "data/processed/border_optics_master_villages.csv"

S2_COLLECTION = "COPERNICUS/S2_SR_HARMONIZED"
VIIRS_COLLECTION = "NOAA/VIIRS/DNB/MONTHLY_V1/VCMSLCFG"

# same summer window as everywhere else
BEFORE = ("2021-06-01", "2021-10-01")
AFTER = ("2025-06-01", "2025-10-01")
```

Purpose: Notice karo — koi `BUFFER_RADIUS_M` constant nahi hai yahan, kyunki iska poora point hi ye hai ki buffer-radius *bada*ta hai, ek fixed value nahi hai. Sirf summer window hardcoded hai (comment khud kehta hai — 'same summer window as everywhere else'), kyunki Entry 13 ke apne reasoning ke mutaabik, sirf summer window hi wo hai jiske paas significant result hai jo stress-test karne layak hai. Syntax: String constants, tuples — familiar patterns, sirf koi buffer-constant nahi hai is baar.

`init_ee()`, `mask_s2_clouds()`, `ndbi_for_period()`, `lights_for_period()` — char shared functions, 'Shared Skeleton' mein poori detail mein dekhi ja chuki hain, character-by-character identical.

extract_buffer() — Ab Buffer Khud Ek Parameter Hai

```
def extract_buffer(buffer_m, checkpoint_every=10):
    out_path = f"data/processed/border_optics_buffer{buffer_m}_summer.csv"

    outcome_cols = [
        "ndbi_before", "ndbi_after", "ndbi_before_image_count", "ndbi_after_image_count",
        "lights_before", "lights_after", "lights_before_image_count",
        "lights_after_image_count",
    ]

    # resume from the checkpoint csv, not the master list, or we lose all progress
    if os.path.exists(out_path):
        villages = pd.read_csv(out_path)
        print(f"Resuming from existing checkpoint: {out_path}")
    else:
        villages = pd.read_csv(VILLAGES_PATH)
```

Purpose: Ye function ka naam khud dikhaata hai iska naya idea — `buffer_m` ab ek fixed constant nahi, ek *parameter* hai, jo caller decide karta hai. `out_path` khud dynamically banta hai, buffer-size ko file-naam mein embed karke (`border_optics_buffer250_summer.csv` ya `buffer1000_summer.csv`) — taaki har buffer-size ki apni, alag file ho. Resume-pattern yahan File 1-6 se alag, simpler hai — coordinate-mismatch-check nahi, sirf PDF 8 ke original `extract_satellite_data.py` jaisa 'agar checkpoint exist karti hai, use hi trust karo' pattern, kyunki ye *treated* villages ke liye hai, jo list stable rehti hai (control ki tarah regenerate nahi hoti). Syntax: F-string se file-path dynamically banana, ek parameter (`buffer_m`) ko embed karke — `f"...buffer{buffer_m}_summer.csv"`. Baaki if/else resume-pattern, PDF 8 ke shuru mein dekha gaya style.

```
for i, row in villages.iterrows():
    if all(pd.isna(row.get(c)) for c in outcome_cols):
        continue

    point = ee.Geometry.Point([row["longitude"], row["latitude"]])
    buffered_geom = point.buffer(buffer_m)

    ndbi_before, ndbi_before_n = ndbi_for_period(buffered_geom, *BEFORE)
    ndbi_after, ndbi_after_n = ndbi_for_period(buffered_geom, *AFTER)
    lights_before, lights_before_n = lights_for_period(buffered_geom, *BEFORE)
    lights_after, lights_after_n = lights_for_period(buffered_geom, *AFTER)
```

Purpose: Yahi is file ka core-difference hai — `point.buffer(buffer_m)`, hardcoded 500 ki jagah, parameter use karta hai. Baaki sab (four function-calls, before/after ke liye) PDF 8/9 jaisa hi, sirf ab har call is naye, variable-size buffer par ho raha hai. Syntax: `point.buffer(buffer_m)` — parameter ko function-argument ki tarah pass karna, hardcoded number ki jagah. `*BEFORE/*AFTER` — wahi tuple-unpacking jo PDF 9 mein dekha.

```

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Extract NDBI/VIIRS at an alternative
buffer radius (summer window).")
    parser.add_argument("--buffer", type=int, choices=[250, 1000], required=True)
    args = parser.parse_args()

    init_ee()
    extract_buffer(args.buffer)

```

Purpose: --buffer flag, jo sirf 250 ya 1000 accept karta hai (500 primary hai, already extract ho chuka hai). type=int naya hai — poore project mein pehli baar, ek argparse flag explicitly ek integer expect kar raha hai, string nahi. Syntax: type=int — argparse ko batana ki command-line se aaya text, ek integer mein convert kiya jaaye (default string hi rehta agar ye na diya jaata). choices=[250, 1000] — is baar integers ki list, strings ki nahi.

EXAMPLE — Entry 12/13 → Entry 31 — Ek Confound Jo Genuinely Reproduce Hua, Fir Genuinely Nagannya Nikla

Is file ki history do rounds mein hai. Entry 12/13 mein, jab ye pehli baar chalayi gayi (500m primary extraction ke hafton baad), 250m aur 1km dono null nikle ($p=0.126$, $p=0.899$), jabki 500m significant tha ($p<0.000001$) — laga jaise result buffer-radius-specific hai. Lekin check karne par, ek asli confound mila: 500m mein 97 villages null thi jo 250m/1km mein nahi thi — aur Sentinel-2 ki tile-footprints itni badi hoti hain ki ek buffer-radius change se scene-availability itna nahi badalna chahiye. Real wajah: teeno extractions alag-alag dinon par hui thi, aur Sentinel-2 ka archive purane dates ke liye bhi scenes backfill karta rehta hai (wahi Entries 21-22 wali kahani, ek doosri jagah). Sample ko 154-village matched-subsample tak restrict karne par, teeno radii significant nikle. Entry 31 mein, ek genuine same-day re-extraction hui — teeno radii (500m primary, 250m, 1000m ye file se) ek hi din. Result: is baar poora clean null teeno radii par (250m $p=0.121$, 500m $p=0.436$, 1km $p=0.898$), full 251-village sample par (koi coverage-gap nahi). Confound real tha principle mein, lekin practically immaterial nikla — fix karne se numbers barely hile. Ek honest additional-note: NDBI ka mean-change sign hi teeno radii mein consistent nahi hai (+0.00175 at 250m, -0.00090 at 500m, -0.00293 at 1000m) — agar koi real, meaningful built-up-change hoti, alag buffer-sizes ko kam-se-kam direction mein toh agree karna chahiye tha.

Ek Chhota Overview — run_all_new_extractions.py

Ye aakhri file, is PDF ki baaki saati files jitni detail nahi maangti — ye khud koi naya extraction-logic nahi likhti, sirf ek 'one-command runner' hai jo File 2 (pre-treatment) aur File 6 (building-footprints) ke saare group/window combinations ko, ek hi command se, ek ke baad ek, chalata hai. Docstring khud iska poora purpose bata deta hai: 'so I don't have to type each one out. If one combo fails it just moves to the next.'

```
def run_pretreatment():
    import extract_pretreatment_baseline as mod
    mod.init_ee()
    results = []
    for group, window in PRETREATMENT_COMBOS:
        try:
            mod.extract(group, window)
            results.append((group, window, "OK"))
        except Exception as e:
            print(f"!! FAILED: group={group} window={window}: {e}")
            results.append((group, window, f"FAILED: {e}"))
        time.sleep(1)
    return results
```

Purpose: Ye function, File 2 ki module ko *import* karke, uske `extract()` function ko har (group, window) combination ke liye call karta hai — ek loop mein, ek try/except ke andar, taaki agar ek combination fail ho (jaise Earth Engine ka temporary error), poora run rukta nahi, agli combination par chala jaata hai. Har result (success ya failure) ek list mein collect hota hai, aakhir mein print karne ke liye. `run_footprints()`, isi pattern ko File 6 ke liye repeat karta hai (yahan nahi dikhaya gaya, kyunki structure hu-ba-hu same hai, sirf module aur combinations badalte hain). **Syntax:** `import extract_pretreatment_baseline as mod` — ek doosri Python file ko, runtime par, ek module ki tarah import karna, taaki uske functions (`mod.extract(...)`) call kiye ja sakein. `try/except Exception as e:` — error ko pakadna aur usko ek variable (e) mein store karna, taaki uska message print ho sake. List of tuples build karna, har iteration mein `.append(...)` se.

`main()` function, `--only` flag (pretreatment ya footprints, ya default dono) ke hisaab se in do functions ko call karta hai, aur aakhir mein ek summary-table print karta hai — kitne combinations OK the, kitne FAILED. Ye file khud koi naya extraction-concept nahi sikhaati, isliye is PDF ka focus, jaisa structure mein decide kiya gaya tha, saati asli extraction-files par raha, ye sirf ek convenience-wrapper hai unke upar.

Synthesis — Saat Files, Ek Bada Sawaal

Is poori PDF ka core sawaal wahi tha jo PDF 8/9 mein tha — 'kya VVP-I ne satellite se dikhne wala built-up-area/lights badla' — lekin ab saat naye angles se poocha gaya. Neeche har check ka final, current (721-village-list) verdict hai.

Check	Result	Verdict
Multi-year trend (2021/23/25)	Wilcoxon on slopes, $p=0.442$ summer	Null (dip-then-rise, not steady)
NDBI/lights placebo (2019→2021)	3 of 4 clean; summer NDBI $p=0.074$	Mostly clean, 1 borderline
Dynamic World placebo (2019→2021)	$p=0.934$ full-year, $p=0.179$ summer	Clean — no pre-trend
SAR VV/VH (sensor-independent)	$p=0.72-0.95$, both bands/windows	Null — strongest confirmation
Dynamic World (algorithm-only)	$p=0.0159$ FY, $p=0.0006$ summer	Significant — still unresolved
Building footprints (ground truth)	ρ 0.40/0.48/0.81 vs NDBI/lights/DW	DW tracks buildings best
Buffer sensitivity (250/500/1000m)	$p=0.121/0.436/0.898$, same-day	Null at every radius

Chhah mein se chhah naye checks — multi-year, dono placebo tests, SAR, building-footprints, buffer-sensitivity — is study ke primary NDBI-based null ko support karte hain, ya kam-se-kam usse contradict nahi karte. Sirf ek — Dynamic World ka apna 2021-vs-2025 result — abhi bhi disagree karta hai, aur is PDF ke dono placebo-tests (NDBI ka bhi, DW ka bhi) us disagreement ko ek 'purana, pehle-se-chal- raha trend' ke roop mein explain nahi karte — dono placebo tests clean hain. Matlab, Dynamic World ka effect, chahe real ho ya ek classifier-specific artifact, genuinely naya hai, treatment ke baad ka.

Ek Bug Pattern Jo Teen Baar Dohraya — Aur Har Baar Sahi Pakda Gaya

Is PDF ki sabse zaroori engineering-kahani, Entry 37 ka coordinate-mismatch-check bug tha — teen jagah (Dynamic World, SAR, building-footprints) missing tha, ek hi jagah (control satellite extraction) present tha. Jab control-list 732 se 721 hui, sirf wo teen jagah stale data reuse kar sakti thi — aur do (Dynamic World, SAR) ne asal mein aisa kiya, chup-chaap. Ye pakda gaya kyunki har rerun ke baad explicitly console-output check kiya gaya ('kitni villages process hui, kya wo sahi count hai'), na

ki sirf ye maan liya gaya ki script chal gayi toh sab theek hai. Ye is poore project ka sabse consistent lesson hai — jo humne PDF 7, 8, 9 mein bhi dekha: ek checkpoint file ka exist karna, iska matlab nahi ki uske peeche ka kaam sahi hua.

Is PDF mein humne dekha: saat files, ek shared skeleton reuse karti hain (init_ee, mask_s2_clouds, ndbi_for_period, lights_for_period, aur coordinate-based resume-with-mismatch-check); multi-year extraction ne Sikkim ke archive-backfill bug ko pakadne mein madad ki aur ek dip-then-rise trend dikhaya; do pre-treatment placebo tests (NDBI/lights, aur alag se Dynamic World) treatment-se-pehle ke parallel-trends assumption ko test karte hain — pehla mostly clean ek borderline exception ke saath, doosra poora clean; SAR (Sentinel-1 radar) is study ka sabse strong, genuinely sensor-independent null-confirmation hai; Dynamic World abhi bhi ek chhota, significant, unexplained disagreement rakhta hai; building-footprints ne ek real ground-truth (Walong/Kaho/Musai) diya jahan NDBI ulti direction mein gaya despite confirmed construction; buffer-sensitivity ne ek genuine archive-timing confound dhoonda aur usse resolve kiya, same-day re-extraction se; aur ek hi resume-logic bug, teen files mein missing tha, Entry 37 mein systematically dhoonda aur fix kiya gaya.

Agla PDF (PDF 11) Core Analysis shuru karega — H1 test (analyze_results.py) aur border-distance computation, jisme us famous 154-village incomplete-sample reversal ki पूरी kahani hai. Jab bhi tayyar ho, bata dena.